SUNWAY UNIVERSITY

CSC3209 Software architecture and Design Patterns

Lecture10: Documenting Software Architectures

Subject Lecturer: Dr. Muhammed Basheer Jasser

References

- Materials from
  - Chapter 18: Len Bass, Paul Clements, Rick Kazman, Software Architecture in Practice, 3/E

Lecture Outline

- Uses and Audiences for Architecture Documentation
- Notations for Architecture Documentation
- Views
- Choosing the Views
- Combining Views
- Building the Documentation Package
- Documenting Behavior
- Architecture Documentation and Quality Attributes
- Documenting Architectures That Change Faster Than You Can Document Them
- Documenting Architecture in an Agile Development Project
- Summary

Architecture Documentation

- Even the best architecture will be useless if the people who need it
  - do not know what it is;
  - cannot understand it well enough to use, build, or modify it;
  - misunderstand it and apply it incorrectly.
- All of the effort, analysis, hard work, and insightful design on the part of the architecture team will have been wasted.

Uses and Audience for Architecture Documentation

- Architecture documentation must
  - be sufficiently transparent and accessible to be quickly understood by new employees
  - be sufficiently concrete to serve as a blueprint for construction
  - have enough information to serve as a basis for analysis.

- Architecture documentation is both prescriptive and descriptive.
  - For some audiences, it prescribes what *should* be true, placing constraints on decisions yet to be made.
  - For other audiences, it describes what *is* true, recounting decisions already made about a system's design.

- Understanding stakeholder uses of architecture documentation is essential
- Those uses determine the information to capture.

Three Uses for Architecture Documentation

Education

- Introducing people to the system
  - New members of the team
  - External analysts or evaluators
  - New architect

Primary vehicle for communication among stakeholders

- Especially architect to developers
- Especially architect to future architect!

Basis for system analysis and construction

- Architecture tells implementers what to implement.
- Each module has interfaces that must be provided and uses interfaces from other modules.
- Documentation can serve as a receptacle for registering and communicating unresolved issues.
- Architecture documentation serves as the basis for architecture evaluation.

Notations

- *Informal notations*
- *Semiformal notations*
- *Formal notations*

Notations

- *Informal notations*
  - Views are depicted (often graphically) using general-purpose diagramming and editing tools and visual conventions chosen for the system at hand.
  - The semantics of the description are characterized in natural language
  - They cannot be formally analyzed
  - The most common tool for informal notations is PowerPoint.

- *Semiformal notations*
  - Views are expressed in a standardized notation that prescribes graphical elements and rules of construction
  - Lacks a complete semantic treatment of the meaning of those elements
  - Rudimentary analysis can be applied to determine if a description satisfies syntactic properties.
  - UML is a semiformal notation in this sense.

Notations

- *Formal notations*
  - Views are described in a notation that has a precise (usually mathematically based) semantics.
  - Formal analysis of both syntax and semantics is possible.
  - Architecture description languages (ADLs)
  - Support automation through associated tools.

$$\forall x \cdot (x \in Train \land TrainLocation(x) = KLCentral) \Rightarrow (TrainSpeed(x) \in \mathbb{N} \land TrainSpeed < 40)$$

Comparison Among the Notations

Notation/Criteria	Preciseness of Specification	Analysis Support	Notation/Language Examples	Tools	Industrial Domains Examples
Informal	Low	Low	Informal diagrams, English	PowerPoint, Informal Diagram Editors	System aspects that do not require specification of critical properties (e.g. foodpanda)
Semi-formal	Medium	Medium	UML	UML Diagram Editors	
Formal	High	High	B-Method, Event-B, Z Method,	Atelier-B [1], Rodin [2]	Safety-critical systems (e.g. control system of autonomous Train: Paris_Métro_Line_14 that uses B-Method for formal verification [3])

[1] https://www.atelierb.eu/en/
[2] http://www.event-b.org/
[3] https://en.wikipedia.org/wiki/Paris_M%C3%A9tro_Line_14

Choosing a Notation

- Tradeoffs
  - Typically, more formal notations take more time and effort to create and understand, but offer reduced ambiguity and more opportunities for analysis.
  - Conversely, more informal notations are easier to create, but they provide fewer guarantees.
- Different notations are better (or worse) for expressing different kinds of information.
  - UML class diagram will not help you reason about safety, nor will a sequence chart tell you very much about the system's likelihood of being delivered on time.
  - Choose your notations and representation languages knowing the important issues you need to capture and reason about.

Views

- Views let us divide a software architecture into a number of (we hope) interesting and manageable representations of the system.

- Principle of architecture documentation:
  - *Documenting an architecture is a matter of documenting the relevant views and then adding documentation that applies to more than one view.*

Which Views? The Ones You Need!

- Different views support different goals and uses.
- We do not advocate a particular view or collection of views.
- The views you should document depend on the uses you expect to make of the documentation.
- Each view has a cost and a benefit; you should ensure that the benefits of maintaining a view outweigh its costs.

Module Views

- A module is an implementation unit that provides a coherent set of responsibilities.
- A module might take the form of a class, a collection of classes, a layer, an aspect, or any decomposition of the implementation unit.

- Example module views are decomposition, uses, and layers.

- Every module has a collection of properties assigned to it.
  - These properties are intended to express the important information associated with the module, as well as constraints on the module.
  - Sample properties are responsibilities, visibility information, and revision history.

- The relations that modules have to one another include is part of, depends on, and is a.

Module Views

- The way in which a system's software is decomposed into manageable units remains one of the important forms of system structure. At a minimum, this determines:
  - how a system's source code is decomposed into units,
  - what kinds of assumptions each unit can make about services provided by other units,
  - and how those units are aggregated into larger ensembles.
- It also includes global data structures that impact and are impacted by multiple units.
- Module structures often determine how changes to one part of a system might affect other parts
  - and hence the ability of a system to support modifiability, portability, and reuse.

Overview of Module Views

- Elements
  - Modules, which are implementation units of software that provide a coherent set of responsibilities.
- Relations
  - *Is part of*, which defines a part/whole relationship between the submodule—the part—and the aggregate module—the whole.
  - *Depends on*, which defines a dependency relationship between two modules. Specific module views elaborate what dependency is meant.
  - *Is a*, which defines a generalization/specialization relationship between a more specific module—the child—and a more general module—the parent.

Overview of Module Views

- Constraints
  - Different module views may impose specific topological constraints, such as limitations on the visibility between modules.

- Usage
  - Blueprint for construction of the code
  - Change-impact analysis
  - Planning incremental development
  - Requirements traceability analysis
  - Communicating the functionality of a system and the structure of its code base
  - Supporting the definition of work assignments, implementation schedules, and budget information
  - Showing the structure of information that the system needs to manage

Module Views: Module Properties

- Properties of modules that help to guide implementation or are input to analysis should be recorded as part of the supporting documentation for a module view.
- The list of properties may vary but is likely to include the following:
  - Name.
  - Responsibilities.
  - Visibility of interface(s).
  - Implementation information
    - Mapping to source code units
    - Test information
    - Management information
    - Implementation constraints
    - Revision history

Module Views: Module Properties

- The list of properties may vary but is likely to include the following:
  - Name.
  - A module's name is, of course, the primary means to refer to it.
  - A module's name often suggests something about its role in the system.
  - In addition, a module's name may reflect its position in a decomposition hierarchy; the name A.B.C, for example, refers to a module C that is a submodule of a module B, itself a submodule of A.

Module Views: Module Properties

- The list of properties may vary but is likely to include the following:
  - Responsibilities.
  - The responsibility property for a module is a way to identify its role in the overall system and establishes an identity for it beyond the name.
  - Whereas a module's name may suggest its role, a statement of responsibility establishes it with much more certainty.
  - Responsibilities should be described in sufficient detail to make clear to the reader what each module does.

Module Views: Module Properties

- The list of properties may vary but is likely to include the following:
  - Visibility of interface(s).
  - When a module has submodules, some interfaces of the submodules are public and some may be private; that is, the interfaces are used only by the submodules within the enclosing parent module.
    - These private interfaces are not visible outside that context.

Module Views: Module Properties

- The list of properties may vary but is likely to include the following:
  - Implementation information.
  - Modules are units of implementation.
  - It is therefore useful to record information related to their implementation from the point of view of managing their development and building the system that contains them. This might include the following:
    - Mapping to source code units
    - Test information
    - Management information
    - Implementation constraints
    - Revision history

Module Views: Module Properties

- The list of properties may vary but is likely to include the following:
  - Implementation information. This might include the following:
    - Mapping to source code units.
    - This identifies the files that constitute the implementation of a module.
    - For example, a module Account, if implemented in Java, might have several files that constitute its implementation:
      - IAccount.java (an interface),
      - AccountImpl.java (the implementation of Account functionality),
      - AccountBean.java (a class to hold the state of an account in memory),
      - AccountOrmMapping.xml (a file that defines the mapping between AccountBean and a database table—object-relational mapping), and
      - perhaps even a unit test AccountTest.java.

Module Views: Module Properties

- The list of properties may vary but is likely to include the following:
  - Implementation information. This might include the following:
    - Test information.
    - The module's test plan, test cases, test scaffolding, and test data are important to document.
    - This information may simply be a pointer to the location of these artifacts..

Module Views: Module Properties

- The list of properties may vary but is likely to include the following:
  - Implementation information. This might include the following:
    - Management information.
    - A manager may need information about the module's predicted schedule and budget.
    - This information may simply be a pointer to the location of these artifacts.

Module Views: Module Properties

- The list of properties may vary but is likely to include the following:
  - Implementation information. This might include the following:
    - Implementation constraints.
      - In many cases, the architect will have an implementation strategy in mind for a module or may know of constraints that the implementation must follow.
    - Revision history. Knowing the history of a module including authors and particular changes may help when you perform maintenance activities.

Module Views: Finer points

- Because modules partition the system, it should be possible to determine how the functional requirements of a system are supported by module responsibilities.

- Module views that show dependencies among modules or layers (which are groups of modules that have a specific pattern of allowed usage) provide a good basis for change-impact analysis.

- Modules are typically modified as a result of problem reports or change requests. Impact analysis requires a certain degree of design completeness and integrity of the module description.

- In particular, dependency information has to be available and correct to be able to create useful results.

Module Views: Finer points

- A module view can be used to explain the system's functionality to someone not familiar with it.
- The various levels of granularity of the module decomposition provide a top-down presentation of the system's responsibilities and therefore can guide the learning process.
- For a system whose implementation is already in place, module views, if kept up to date, are helpful, as they explain the structure of the code base to a new developer on the team.
- Thus, up-to-date module views can simplify and regularize system maintenance.

Module Views: Finer points

- On the other hand, it is difficult to use the module views to make inferences about runtime behavior, because these views are just a static partition of the functions of the software.

- Thus, a module view is not typically used for analysis of performance, reliability, and many other runtime qualities.
  - For those, we rely on component-and-connector and allocation views.

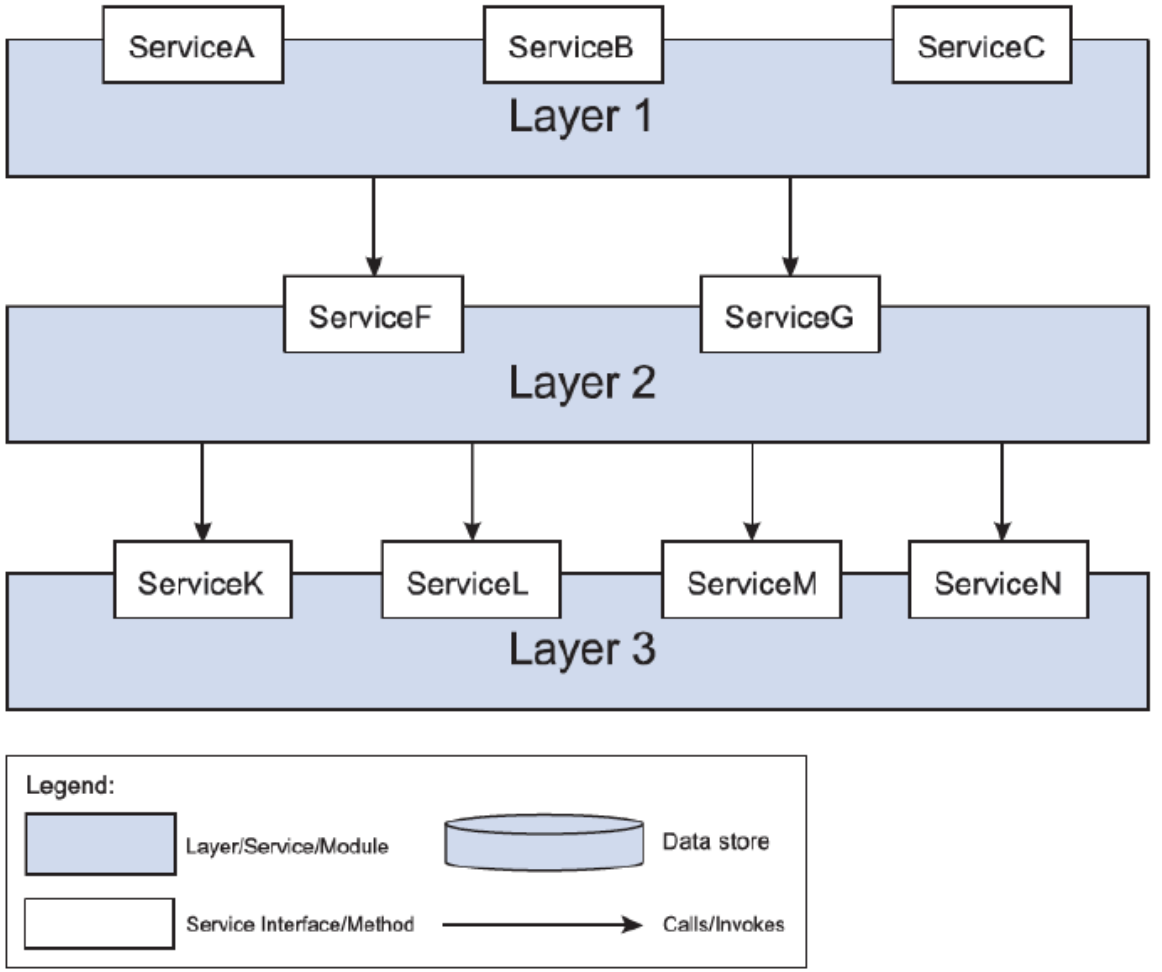
- Module views are commonly mapped to component-and-connector views.
  - The implementation units shown in module views have a mapping to components that execute at runtime.
  - Sometimes, the mapping is quite straightforward, even one-to-one for small, simple applications.
  - More often, a single module will be replicated as part of many runtime components, and a given component could map to several modules.

- Module views also provide the software elements that are mapped to the diverse non-software elements of the system environment in the various allocation views.

Module Views: Conclusion

- It is unlikely that the documentation of any software architecture can be complete without at least one module view.

Module Views: Example



Component-and-Connector Views

- Component-and-connector views show elements that have some runtime presence, such as processes, objects, clients, servers, and data stores.

- Additionally, component-and-connector views include as elements the pathways of interaction, such as communication links and protocols, information flows, and access to shared storage.
  - Such interactions are represented as connectors in C&C views.

- Sample C&C views are service-oriented architecture (SOA), client-server.

Component-and-Connector Views: Components

- A component in a C&C view may represent a complex subsystem, which itself can be described as a C&C sub-architecture.

- This sub-architecture can be depicted graphically in *situ* when the substructure is not too complex, by showing it as nested inside the component that it refines.

- Often, however, it is documented separately. A component's sub-architecture may employ a different pattern than the one in which the component appears.

Component-and-Connector Views: Connectors

- Connectors are the other kind of element in a C&C view.
- Simple examples of connectors are service invocation; asynchronous message queues; event multicast supporting publish-subscribe interactions; and pipes that represent asynchronous, order-preserving data streams.
- Connectors often represent much more complex forms of interaction, such as a transaction-oriented communication channel between a database server and a client, or an enterprise service bus that mediates interactions between collections of service users and providers.

Component-and-Connector Views: Connectors

- Connectors have roles, which are its interfaces, defining the ways in which the connector may be used by components to carry out interaction. For example:
  - A client-server connector might have invokes-services and provides-services roles.
  - A pipe might have writer and reader roles.
  - A publish-subscribe connector might have many instances of the publisher and subscriber roles.

Component-and-Connector Views

- The primary relation within a C&C view is attachment.
- Attachments indicate which connectors are attached to which components, thereby defining a system as a graph of components and connectors.

Component-and-Connector Views

- The primary relation within a C&C view is attachment.
- Attachments indicate which connectors are attached to which components, thereby defining a system as a graph of components and connectors.

Component-and-Connector Views: Element Properties

- Every element (component or connector) should have a name and type.
- Additional properties depend on the type of component or connector.
- Define values for the properties that support the intended analyses for the particular C&C view.
  - For example, if the view will be used for performance analysis, latencies, queue capacities, and thread priorities may be necessary.

Component-and-Connector Views: Element Properties

- The following are examples of some typical properties and their uses:
- Reliability. What is the likelihood of failure for a given component or connector?
  - This property might be used to help determine overall system availability.

- Performance. What kinds of response time will the component provide under what loads? What kind of bandwidth, latency, jitter, transaction volume, or throughput can be expected for a given connector?
  - This property can be used with others to determine system-wide properties such as response times, throughput, and buffering needs.

- Resource requirements. What are the processing and storage needs of a component or a connector?
  - This property can be used to determine whether a proposed hardware configuration will be adequate.

Component-and-Connector Views: Element Properties

- The following are examples of some typical properties and their uses:
- Functionality. What functions does an element perform?
  - This property can be used to reason about overall computation performed by a system.
- Security. Does a component or a connector enforce or provide security features, such as encryption, audit trails, or authentication?
  - This property can be used to determine system security vulnerabilities.
- Concurrency. Does this component execute as a separate process or thread?
  - This property can help to analyze or simulate the performance of concurrent components and identify possible deadlocks.

Component-and-Connector Views: Element Properties

- The following are examples of some typical properties and their uses:

- Modifiability. Does the messaging structure support a structure to cater for evolving data exchanges? Can the components be adapted to process those new messages?
  - This property can be defined to extend the functionality of a component.

- Tier. For a tiered topology, what tier does the component reside in?
  - This property helps to define the build and deployment procedures, as well as platform requirements for each tier.

Overview of C&C Views

- Elements
  - *Components*. Principal processing units and data stores. A component has a set of *ports* through which it interacts with other components (via connectors).
  - *Connectors*. Pathways of interaction between components. Connectors have a set of roles (interfaces) that indicate how components may use a connector in interactions.
- Relations
  - *Attachments*. Component ports are associated with connector roles to yield a graph of components and connectors.
  - *Interface delegation.* In some situations component ports are associated with one or more ports in an "internal" sub-architecture. The case is similar for the roles of a connector

Overview of C&C Views

- Constraints
  - Components can only be attached to connectors, not directly to other components.
  - Connectors can only be attached to components, not directly to other connectors.
  - Attachments can only be made between compatible ports and roles.
  - Interface delegation can only be defined between two compatible ports (or two compatible roles).
  - Connectors cannot appear in isolation; a connector must be attached to a component.

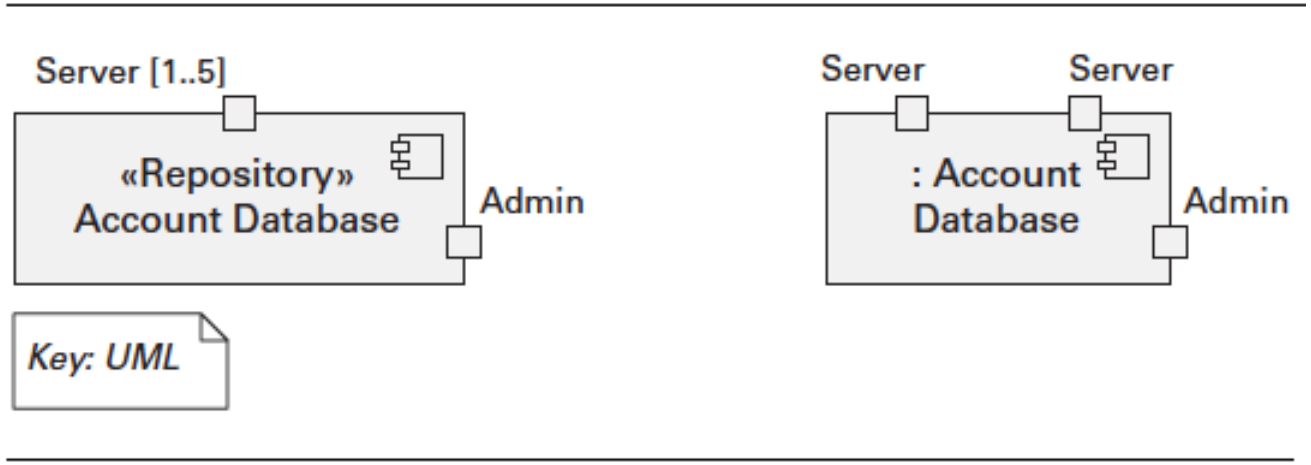
- Usage
  - Show how the system works.
  - Guide development by specifying structure and behavior of runtime elements.
  - Help reason about runtime system quality attributes, such as performance and availability.

Notations for C&C Views

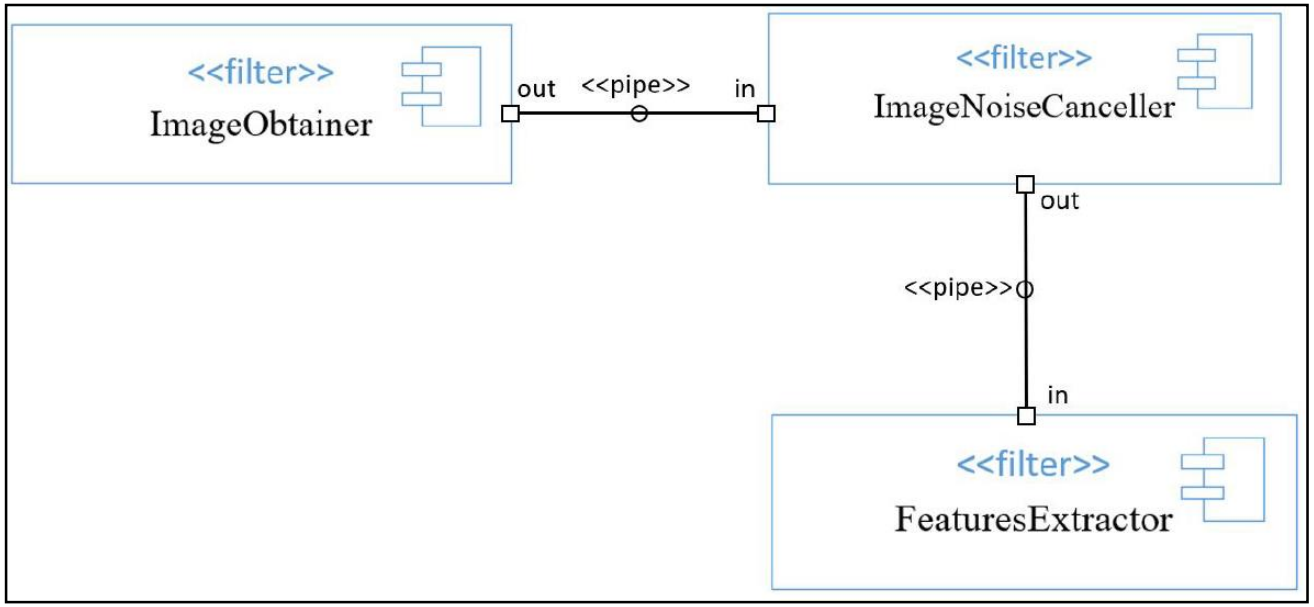
- As always, box-and-line drawings are available to represent C&C views.
- Although informal notations are limited in the semantics that can be conveyed, following some simple guidelines can lend rigor and depth to the descriptions.
- The primary guideline is simple: assign each component type and each connector type a separate visual form (symbol), and list each of the types in a key.

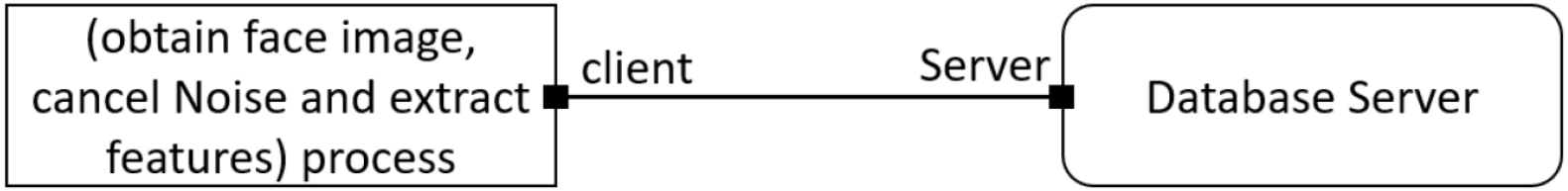
Notations for C&C Views

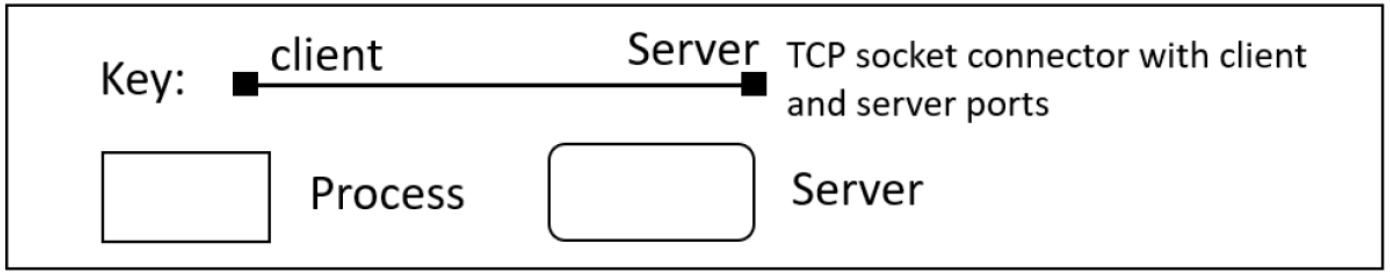
- UML components are a good semantic match to C&C components because they permit intuitive documentation of important information like interfaces, properties, and behavioral descriptions.
- UML components also distinguish between component types and component instances, which is useful when defining view-specific component types.
- You should represent a “simple” C&C connector using a UML connector—a line.
- UML ports are a good semantic match to C&C ports. A UML port can be decorated with a multiplicity, as shown in the left portion of the Figure.



Notations for C&C Views: Example







Allocation Views

- Allocation views describe the mapping of software units to elements of an environment in which the software is developed or in which it executes.

- The software elements come from a module or C&C view.

- The environment might be the hardware, the operating environment in which the software is executed, the file systems supporting development or deployment, or the development organization(s).
  - Examples of environmental elements are a processor, a disk farm, a file or folder, or a group of developers.

Allocation Views

- The relation in an allocation view is allocated to.
- A single software element can be allocated to multiple environmental elements, and multiple software elements can be allocated to a single environmental element.
- If these allocations change over time, either during development or execution of the system, then the architecture is said to be dynamic with respect to that allocation.
- For example, processes might migrate from one processor or virtual machine to another. Similarly modules might migrate from one development team to another.

Allocation Views: Elements Properties

- Software elements and environmental elements have properties in allocation views.
- The usual goal of an allocation view is to compare the properties required by the software element with the properties provided by the environmental elements to determine whether the allocation will be successful or not.

- For example, to ensure a component's required response time, it has to execute on (be allocated to) a processor that provides sufficiently fast processing power.
- For another example, a computing platform might not allow a task to use more than 10 kilobytes of virtual memory. An execution model of the software element in question can be used to determine the required virtual memory usage.
- Similarly, if you are migrating a module from one team to another, you might want to ensure that the new team has the appropriate skills and background knowledge.

Allocation Views: Static vs Dynamic

- Allocation views can depict static or dynamic views.
- A static view depicts a fixed allocation of resources in an environment.
- A dynamic view depicts the conditions and the triggers for which allocation of resources changes according to loading.
- Some systems recruit and utilize new resources as their load increases.
  - An example is a load-balancing system in which new processes or threads are created on another machine.
  - In this view, the conditions under which the allocation changes, the allocation of runtime software, and the dynamic allocation mechanism need to be documented. (Recall from Lecture1 that one of the allocation structures is the work assignment structure, which allocates modules to teams for development.
- That relationship, too, can be allocated dynamically, depending on "load"—in this case, the load on development teams.)

Overview of Allocation Views

- Elements
  - *Software element* and *environmental element*.
  - A software element has properties that are *required* of the environment.
  - An environmental element has properties that are *provided* to the software.

- Relations
  - *Allocated to*. A software element is mapped (allocated to) an environmental element. Properties are dependent on the particular view.

Overview of Allocation Views

- Constraints
  - Varies by view

- Usage
  - Reasoning about performance, availability, security, and safety.
  - Reasoning about distributed development and allocation of work to teams.
  - Reasoning about concurrent access to software versions.
  - Reasoning about the form and mechanisms of system installation.

Quality Views

- Module, C&C, and allocation views are all structural views:
  - They primarily show the structures that the architect has engineered into the architecture to satisfy functional and quality attribute requirements.
- These views are excellent for guiding and constraining downstream developers, whose primary job it is to implement those structures.
- However, in systems in which certain quality attributes (or, for that matter, some other kind of stakeholder concerns) are particularly important and pervasive, structural views may not be the best way to present the architectural solution to those needs.
- The reason is that the solution may be spread across multiple structures that are inconvenient to combine (for example, because the element types shown in each are different).

Quality Views

- Another kind of view, which we call a quality view, can be tailored for specific stakeholders or to address specific concerns.
- These quality views are formed by extracting the relevant pieces of structural views and packaging them together.
- Here are five examples:
  - A security view
  - A communications view
  - An exception or error-handling view
  - A reliability view
  - A performance view

Quality Views: Examples

- *Security view*
  - Show the components that have some security role or responsibility, how those components communicate, any data repositories for security information, and repositories that are of security interest.
  - The view's context information would show other security measures (such as physical security) in the system's environment.
  - The behavior part of a security view
    - Show how the operation of security protocols and where and how humans interact with the security elements.
    - Capture how the system would respond to specific threats and vulnerabilities.

Quality Views: Examples

- *Communications view*
  - Especially helpful for systems that are globally dispersed and heterogeneous.
  - Show all of the component-to-component channels, the various network channels, quality-of-service parameter values, and areas of concurrency.
  - Used to analyze certain kinds of performance and reliability (such as deadlock or race condition detection).
  - The behavior part of this view could show (for example) how network bandwidth is dynamically allocated.

Quality Views: Examples

- *Exception* or *error-handling view*
  - Could help illuminate and draw attention to error reporting and resolution mechanisms.
  - Show how components detect, report, and resolve faults or errors.
  - It would help identify the sources of errors and appropriate corrective actions for each.
- *Reliability* view
  - Models mechanisms such as replication and switchover.
  - Depicts timing issues and transaction integrity.
- *Performance* view
  - Shows those aspects of the architecture useful for inferring the system's performance.
  - Show network traffic models, maximum latencies for operations, and so forth.

Choosing the Views

- You can determine which views are required, when to create them, and how much detail to include if you know the following:
  - What people, and with what skills, are available
  - Which standards you have to comply with
  - What budget is on hand
  - What the schedule is
  - What the information needs of the important stakeholders are
  - What the driving quality attribute requirements are
  - What the approximate size of the system is

Choosing the Views

- At a minimum, expect to have:
  - at least one module view,
  - at least one C&C view, and
  - for larger systems, at least one allocation view in your architecture document.

- Beyond that basic rule of thumb, however, there is a three-step method for choosing the views:

Method for Choosing the Views

- **Step 1. Build a stakeholder/view table.**
  - Rows: List the stakeholders for your project's software architecture documentation
  - Columns: Enumerate the views that apply to your system.
    - Use the structures discussed in Lecture 1, the views discussed in this lecture, and the views that your design work in ADD has suggested as a starting list of candidates.
    - Include the views or view sketches you have as a result of your design work so far.
  - Some views (such as decomposition, uses, and work assignment) apply to every system, while others (various C&C views, the layered view) only apply to some systems.
  - Fill in each cell to describe how much information the stakeholder requires from the view: none, overview only, moderate detail, or high detail.

Method for Choosing the Views

- **Step 2. Combine views to reduce their number**
  - Look for marginal views in the table; those that require only an overview, or that serve very few stakeholders.
  - Combine each marginal view with another view that has a stronger constituency.

  - These views often combine naturally:
    - *Various C&C views*. Because C&C views all show runtime relations among components and connectors of various types, they tend to combine well.

    - *Deployment view with either SOA* ***or*** *communicating-processes views*. An SOA view shows services, and a communicating-processes view shows processes. In both cases, these are components that are deployed onto processors.

    - *Decomposition view and any of work assignment*, *implementation*, *uses*, ***or*** *layered views*. The decomposed modules form the units of work, development, and uses. In addition, these modules populate layers.

Method for Choosing the Views

- **Step 3. Prioritize and stage**.
  - The decomposition view (one of the module views) is a particularly helpful view to release early.
    - High-level decompositions are often easy to design
    - The project manager can start to staff development teams, put training in place, determine which parts to outsource, and start producing budgets and schedules.
  - You don't have to satisfy all the information needs of all the stakeholders to the fullest extent.
    - Providing 80 percent of the information goes a long way, and this might be "good enough" so that the stakeholders can do their job.
    - Check with the stakeholder if a subset of information would be sufficient.
  - You don't have to complete one view before starting another.
    - People can make progress with overview-level information
    - A breadth-first approach is often the best.

Combining Views

- The basic principle of documenting an architecture as a set of separate views brings a divide-and-conquer advantage to the task of documentation,
  - but if the views were irrevocably different, with no association with one another, nobody would be able to understand the system as a whole.

- Because all views in an architecture are part of that same architecture and exist to achieve a common purpose, many of them have strong associations with each other.

- Managing how architectural structures are associated is an important part of the architect's job, independent of whether any documentation of those structures exists.

Combining Views

- Sometimes the most convenient way to show a strong association between two views is to collapse them into a single combined view, as dictated by step 2 of the three-step method just presented to choose the views.

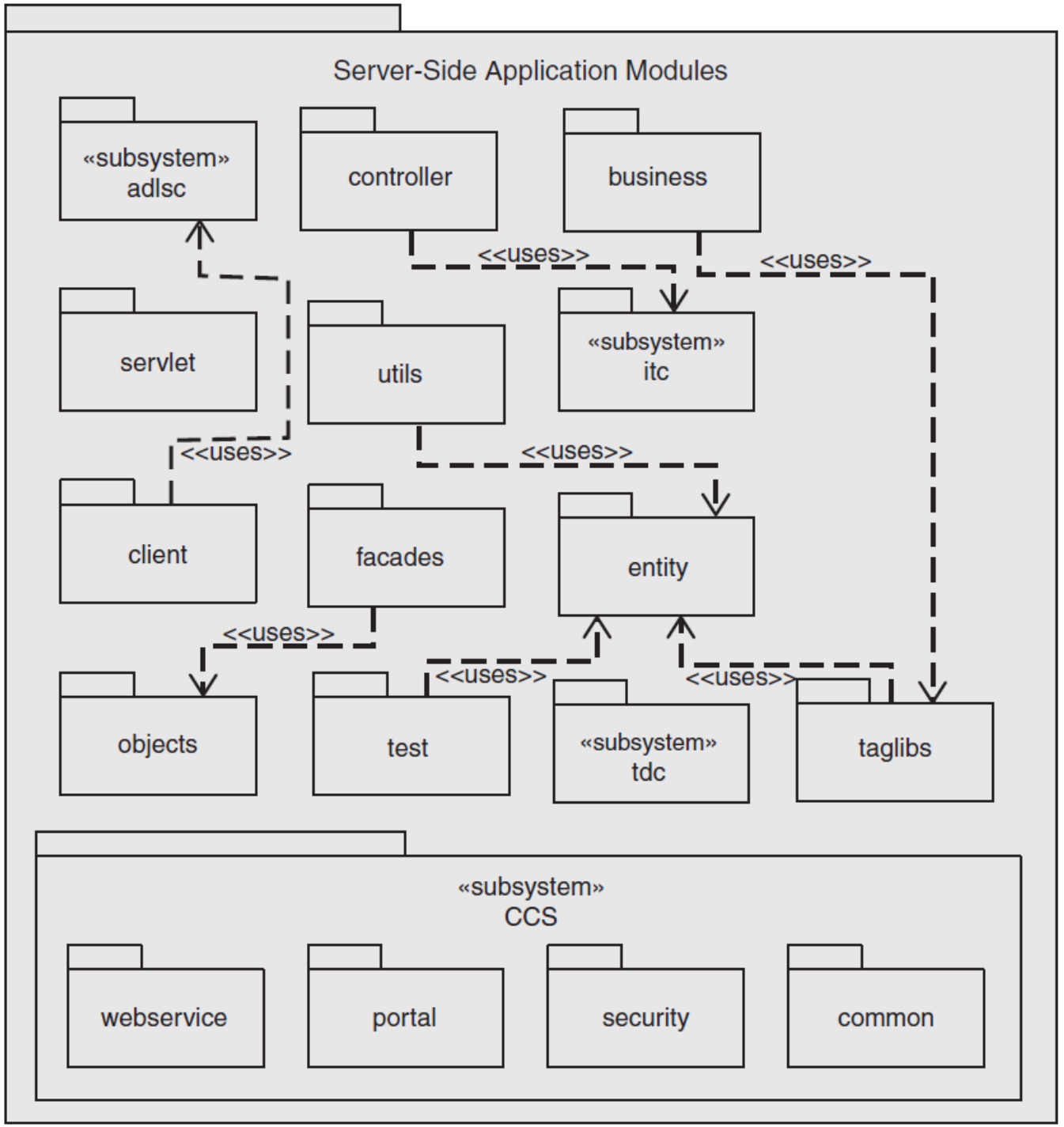
- A combined view is a view that contains elements and relations that come from two or more other views. Combined views can be very useful as long as you do not try to overload them with too many mappings.

- The easiest way to merge views is to create an overlay that combines the information that would otherwise have been in two separate views. This works well if the coupling between the two views is tight;
  - that is, there are strong associations between elements in one view and elements in the other view.
  - If that is the case, the structure described by the combined view will be easier to understand than the two views seen separately.

Combining Views

- For an example, see the overlay of decomposition and uses sketches shown in the figure. In an overlay, the elements and the relations keep the types as defined in their constituent views.

- A decomposition view overlaid with “uses” information, to create a decomposition/uses overlay.



Notation: UML

Combining Views

- The views below often combine naturally:
- Various C&C views. Because C&C views all show runtime relations among components and connectors of various types, they tend to combine well.
  - Different (separate) C&C views tend to show different parts of the system, or tend to show decomposition refinements of components in other views. The result is often a set of views that can be combined easily.

- Deployment view with either SOA **or** communicating-processes views. An SOA view shows services, and a communicating-processes view shows processes.
  - In both cases, these are components that are deployed onto processors. Thus there is a strong association between the elements in these views.

- *Decomposition view and any of work assignment, implementation, uses,* ***or*** *layered views.* The decomposed modules form the units of work, development, and uses. In addition, these modules populate layers.

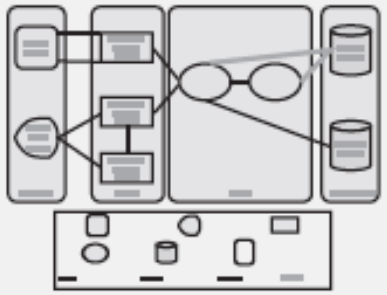
Building the Documentation Package

- Documentation package consists of
  - Views
  - Documentation beyond views

View Template

Template for a View

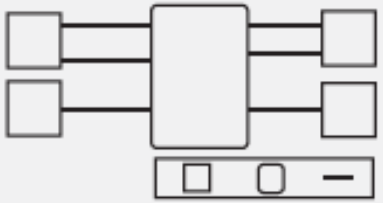
Section 1. Primary Presentation



Section 2. Element Catalog

- Section 2.A. Elements and Their Properties
- Section 2.B. Relations and Their Properties
- Section 2.C. Element Interfaces
- Section 2.D. Element Behavior

Section 3. Context Diagram



Section 4. Variability Guide

Section 5. Rationale

Documenting a View

- **Section 1: The Primary Presentation.**
  - The *primary presentation* shows the elements and relations of the view.
  - The primary presentation should contain the information you wish to convey about the system—in the vocabulary of that view.
  - The primary presentation is most often graphical.
    - It might be a diagram you've drawn in an informal notation using a simple drawing tool, or it might be a diagram in a semiformal or formal notation imported from a design or modeling tool that you're using.
    - If your primary presentation is graphical, make sure to include a key that explains the notation.
    - Lack of a key is the most common mistake that we see in documentation in practice.
  - Occasionally the primary presentation will be textual, such as a table or a list.
    - If that text is presented according to certain stylistic rules, these rules should be stated or incorporated by reference, as the analog to the graphical notation key.

Documenting a View

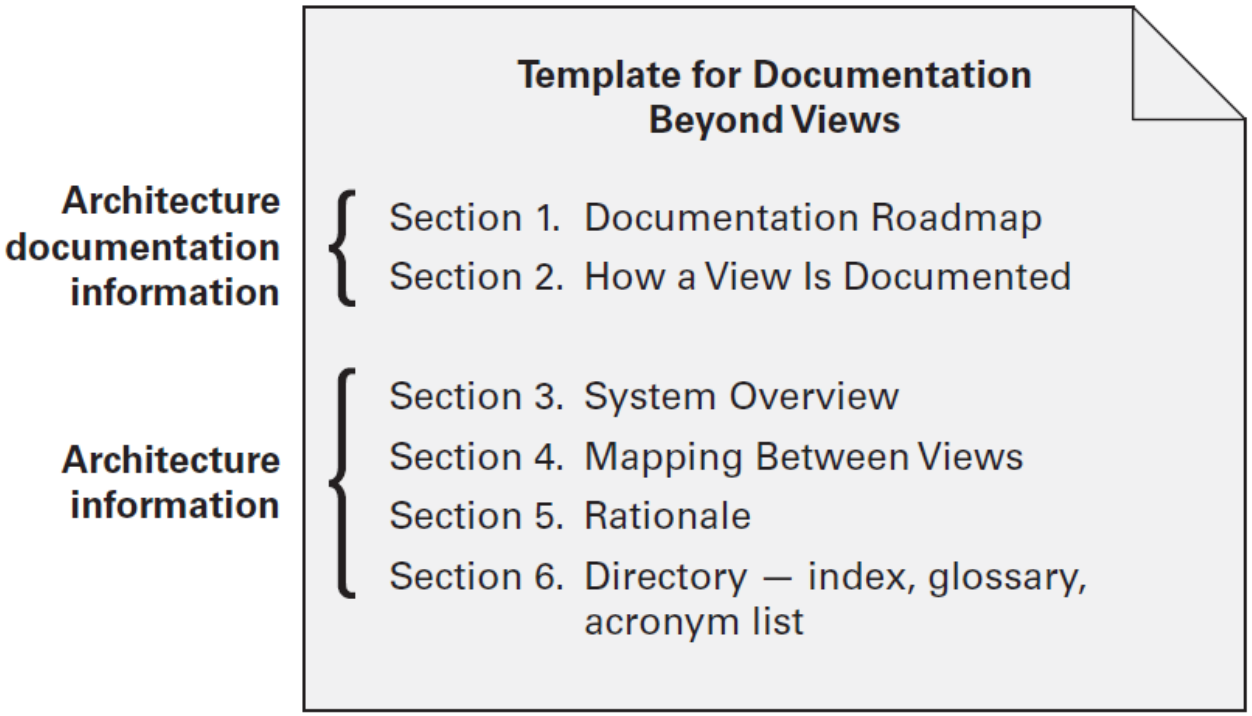
- **Section 2: The Element Catalog.**
  - The *element catalog* details at least those elements depicted in the primary presentation.
    - For instance, if a diagram shows elements A, B, and C, then the element catalog needs to explain what A, B, and C are.
    - If elements or relations relevant to this view were omitted from the primary presentation, they should be introduced and explained in the catalog.
  - Parts of the catalog:
    - *Elements and their properties*. This section names each element in the view and lists the properties of that element. Each view introduced in Lecture 1 listed a set of suggested properties associated with that view.
    - *Relations and their properties*. Each view has specific relation types that it depicts among the elements in that view.
    - *Element interfaces*. This section documents element interfaces.
    - *Element behavior*. This section documents element behavior that is not obvious from the primary presentation.

Documenting a View

- **Section 3: Context Diagram.**
  - A *context diagram* shows how the system or portion of the system depicted in this view relates to its environment.
  - The purpose of a context diagram is to depict the scope of a view.
  - Entities in the environment may be humans, other computer systems, or physical objects, such as sensors or controlled devices.
- **Section 4: Variability Guide.**
  - A *variability guide* shows how to exercise any variation points that are a part of the architecture shown in this view.
- **Section 5: Rationale.**
  - *Rationale* explains why the design reflected in the view came to be.
  - The goal of this section is to explain why the design is as it is and to provide a convincing argument that it is sound.
  - The choice of a pattern in this view should be justified here by describing the architectural problem that the chosen pattern solves and the rationale for choosing it over another.

Documenting Information Beyond Views

- Documentation beyond views can be divided into two parts:
  - Overview of the architecture documentation. This tells how the documentation is laid out and organized so that a stakeholder of the architecture can find the information he or she needs efficiently and reliably.
  - Information about the architecture. Here, the information that remains to be captured beyond the views themselves is a short system overview to ground any reader as to the purpose of the system and the way the views are related to one another, an overview of and rationale behind system-wide design approaches, a list of elements and where they appear, and a glossary and an acronym list for the entire architecture.



Documenting Information Beyond Views Sections

- **Document control information.**
  - List the issuing organization, the current version number, date of issue and status, a change history, and the procedure for submitting change requests to the document.
  - Usually captured in the front matter

Documenting Information Beyond Views Sections

- **Section 1: Documentation Roadmap.** The documentation map tells the reader what information is in the documentation and where to find it.
  - *Scope and summary*. Explain the purpose of the document and briefly summarize what is covered.
  - *How the documentation is organized*. For each section in the documentation, give a short synopsis of the information that can be found there.
  - *View overview*. Describes the views that the architect has included in the package. For each view::
    - The name of the view and what pattern it instantiates, if any.
    - A description of the view's element types, relation types, and property types.
    - A description of language, modeling techniques, or analytical methods used in constructing the view.
  - *How stakeholders can use the documentation*.
    - This section shows how various stakeholders might use the documentation to help address their concerns.
    - Include short scenarios, such as "A maintainer wishes to know the units of software that are likely to be changed by a proposed modification."
    - To be compliant with ISO/IEC 42010-2007, you must consider the concerns of at least users, acquirers, developers, and maintainers.

Documenting Information Beyond Views Sections

- **Section 2: How a View Is Documented.**
  - Explain the standard organization you're using to document views—either the one described in this lecture or one of your own.

- **Section 3: System Overview.**
  - Short prose description of the system's function, its users, and any important background or constraints.
  - Provides your readers with a consistent mental model of the system and its purpose.
  - This might be a pointer to your project's concept-of-operations document for the system.

Documenting Information Beyond Views Sections

- **Section 4: Mapping Between Views.**
  - Helping a reader understand the associations between views will help that reader gain a powerful insight into how the architecture works as a unified conceptual whole.
  - The associations between elements across views in an architecture are, in general, many-to-many.
  - View-to-view associations can be captured as tables.
    - The table should name the correspondence between the elements across the two views.
    - Examples
      - "is implemented by" for mapping from a component-and-connector view to a module view
      - "implements" for mapping from a module view to a component-and-connector view
      - "included in" for mapping from a decomposition view to a layered view

Documenting Information Beyond Views Sections

- **Section 5: Rationale.**
  - Documents the architectural decisions that apply to more than one view.
    - Documentation of background or organizational constraints or major requirements that led to decisions of system-wide import.
    - Decisions about which fundamental architecture patterns are used.

- **Section 6: Directory.**
  - Set of reference material that helps readers find more information quickly.
    - Index of terms
    - Glossary
    - Acronym list.

Documenting Patterns

- Architects can, and typically do, use patterns as a starting point for their design, as we have discussed in Lectuer4.

- These patterns might be published in existing catalogs or in an organization's proprietary repository of standard designs, or created specifically for the problem at hand by the architect.

- In each of these cases, they provide a generic (that is, incomplete) solution approach that the architect will have to refine and instantiate.

Documenting Patterns

- First, record the fact that the given pattern is being used.

- Then say why this solution approach was chosen—why it is a good fit to the problem at hand.
  - If the chosen approach comes from a pattern, this will consist essentially of showing that the problem at hand fits the problem and context of the pattern.

Documenting Patterns

- Using a pattern means making successive design decisions that eventually result in an architecture.

- These design decisions manifest themselves as newly instantiated elements and relations among them.

- The architect can document a snapshot of the architecture at each stage. How many stages there are depends on many things, not the least of which is the ability of readers to follow the design process in case they have to revisit it in the future.

Documenting Behavior

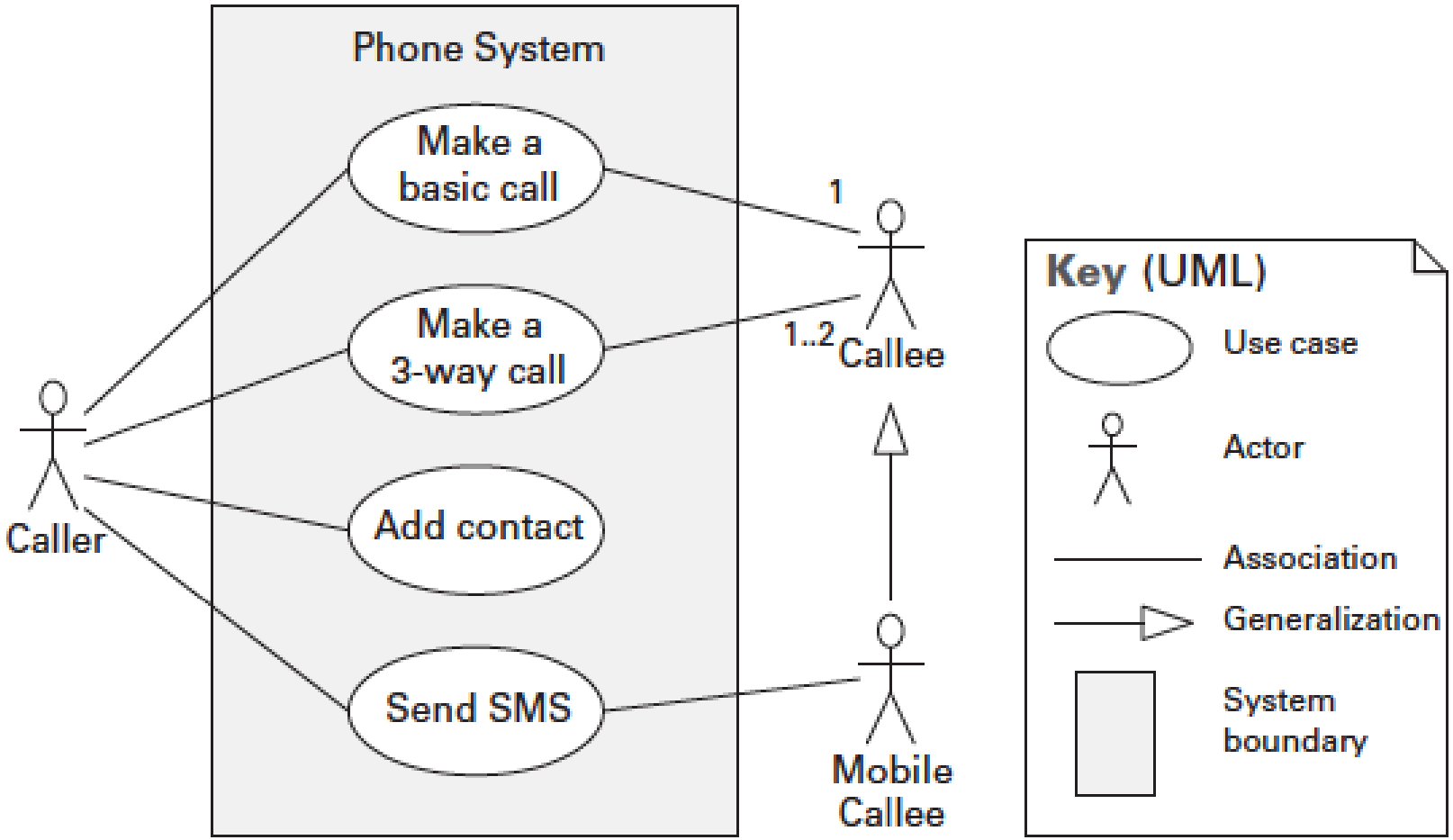
- Behavior documentation complements each views by describing how architecture elements in that view interact with each other.

- Behavior documentation enables reasoning about
  - a system's potential to deadlock
  - a system's ability to complete a task in the desired amount of time
  - maximum memory consumption
  - and more
- Behavior has its own section in our view template's element catalog.

Notations for Documenting Behavior

- Trace-oriented languages
  - *Traces* are sequences of activities or interactions that describe the system's response to a specific stimulus when the system is in a specific state.
  - A trace describes a particular sequence of activities or interactions between structural elements of the system.
  - Examples
    - use cases
    - sequence diagrams
    - communication diagrams
    - activity diagrams
    - message sequence charts
    - timing diagrams
    - Business Process Execution Language

Use Case Diagram



From *Documenting Software Architectures: Views and Beyond*, 2nd edition, Addison Wesley, 2010

Use Case Description

Name: Make a basic call

Description: Making a point-to-point connection between two phones.

Primary actors: Caller

Secondary actors: Callee

Flow of events:

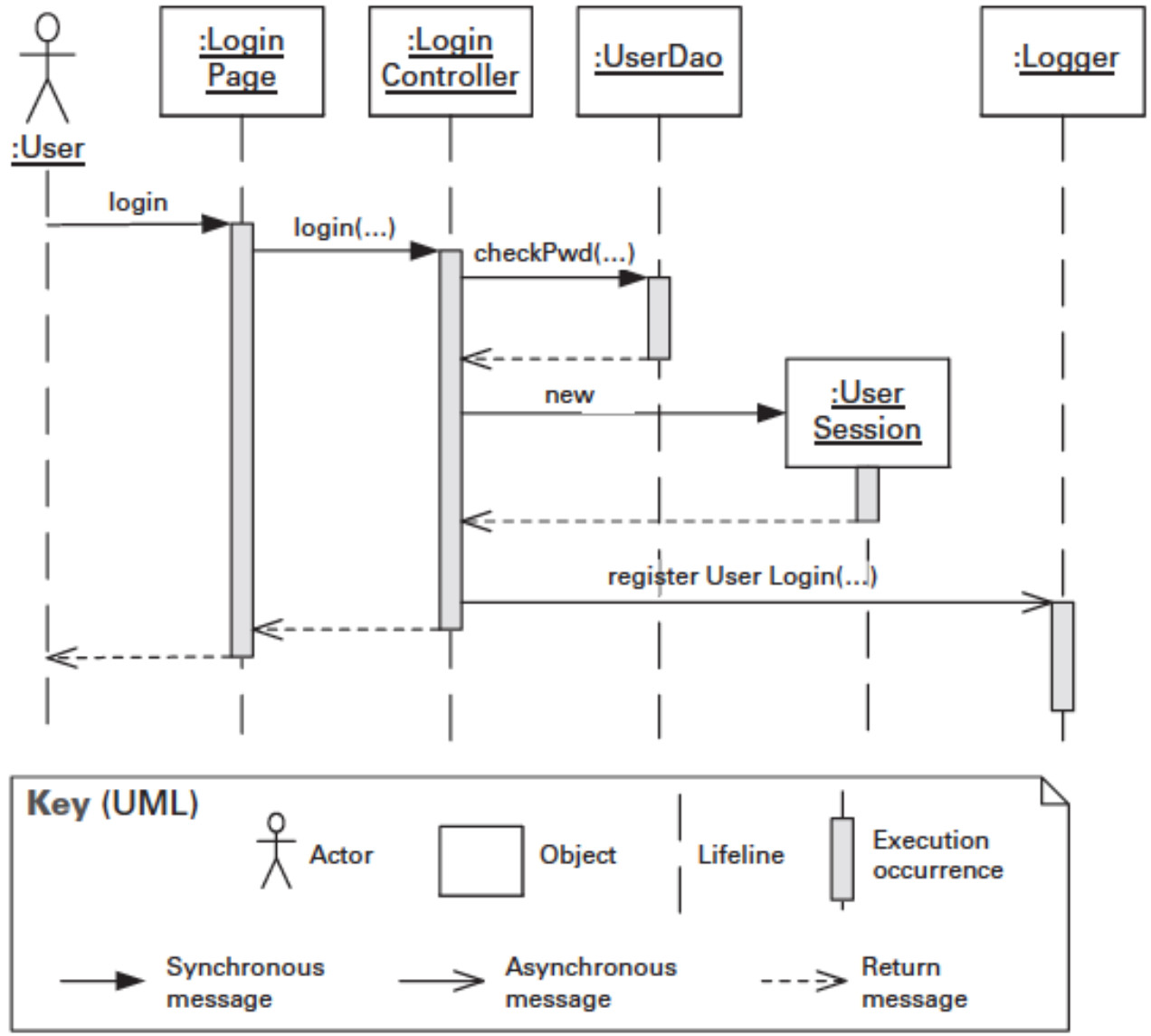
The use case starts when a caller places a call via a terminal, such as a cell phone. All terminals to which the call should be routed then begin ringing. When one of the terminals is answered, all others stop ringing and a connection is made between the caller's terminal and the terminal that was answered. When either terminal is disconnected—someone hangs up—the other terminal is also disconnected. The call is now terminated, and the use case is ended.

Exceptional flow of events:

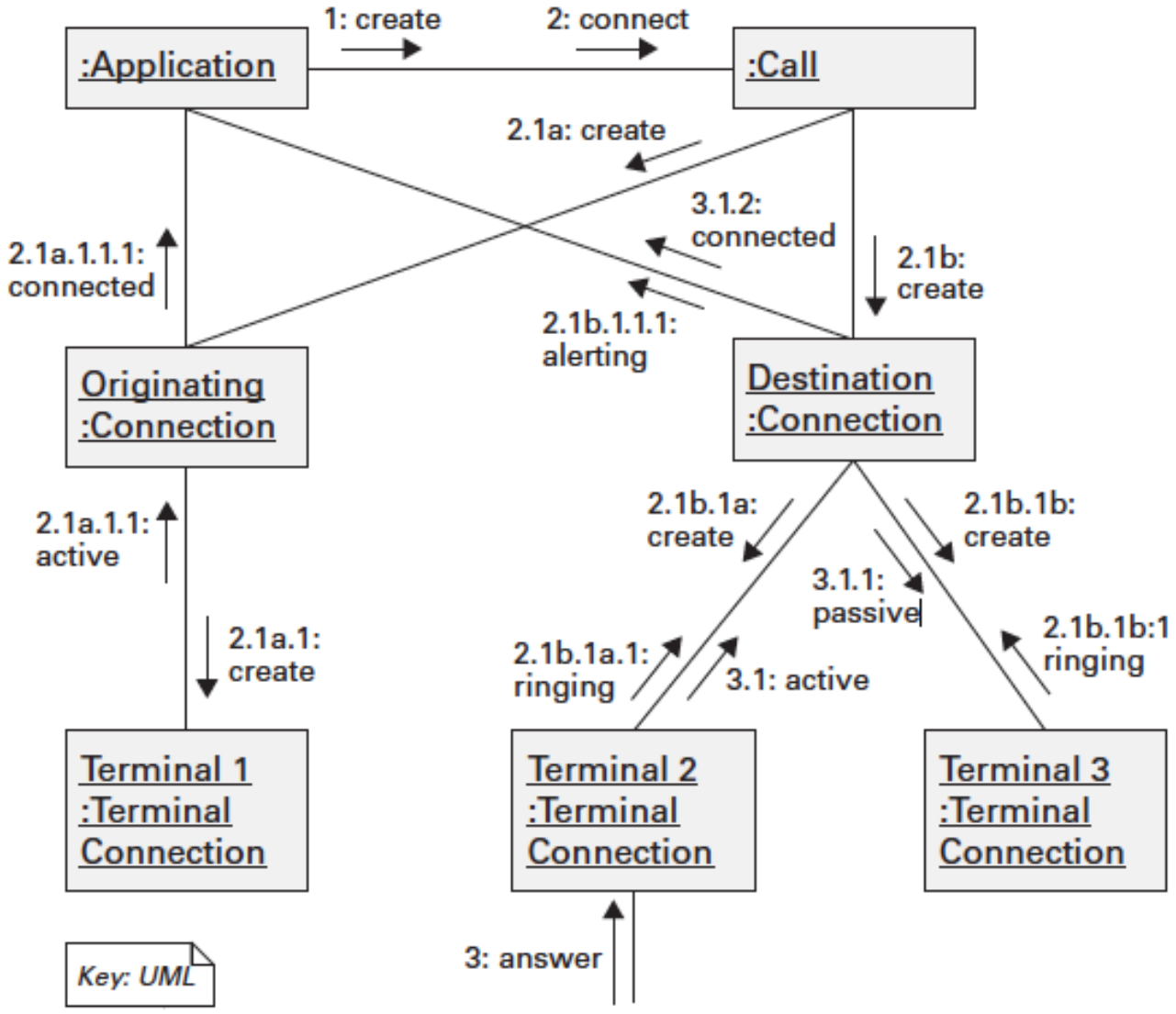
The caller can disconnect, or hang up, before any of the ringing terminals has been answered. If this happens, all ringing terminals stop ringing and are disconnected, ending the use case.

From *Documenting Software Architectures: Views and Beyond*, 2nd edition, Addison Wesley, 2010

Sequence Diagram

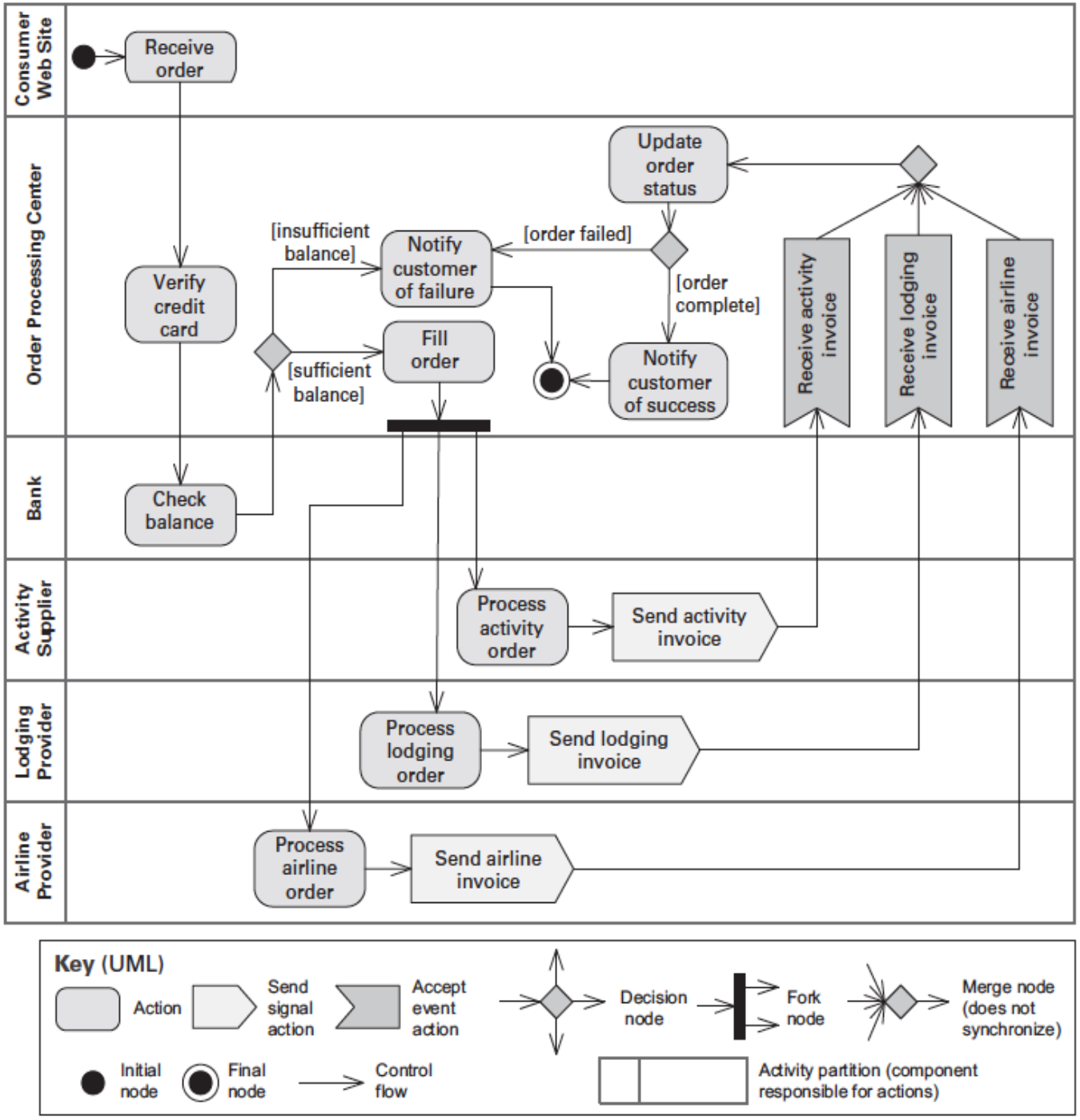


Communication Diagram



From *Documenting Software Architectures: Views and Beyond*, 2nd edition, Addison Wesley, 2010

Activity Diagram

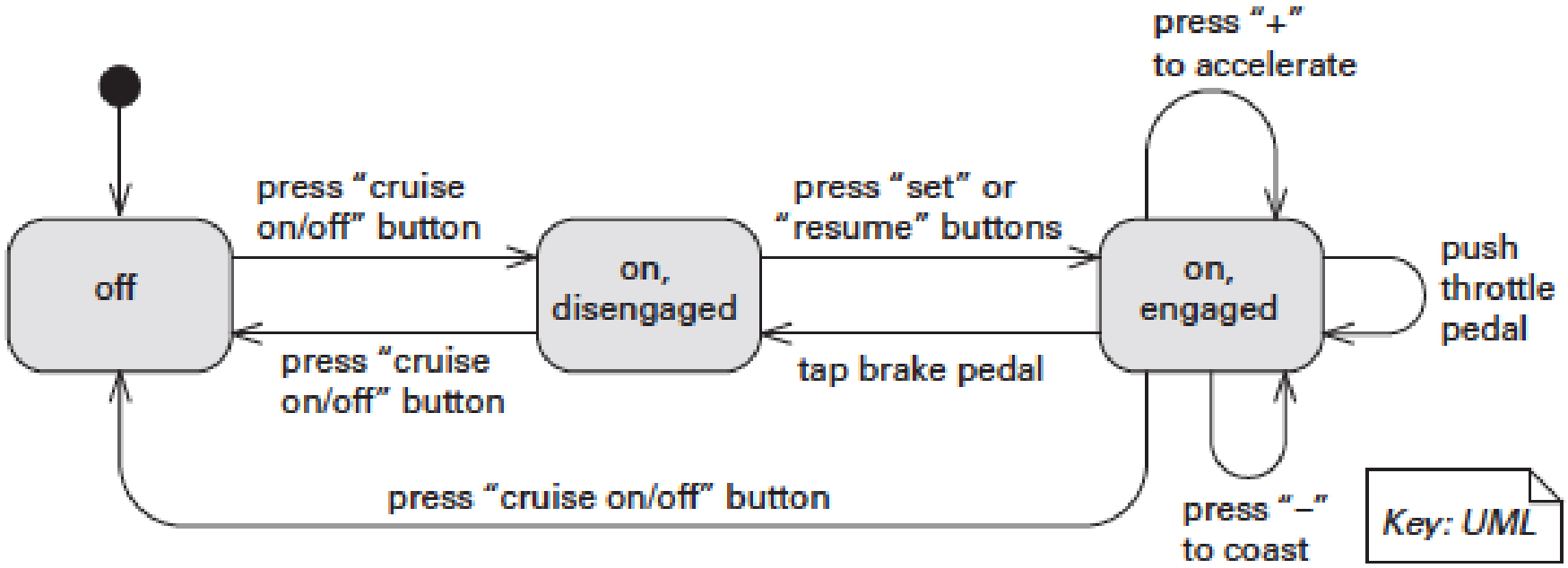


From *Documenting Software Architectures: Views and Beyond*, 2nd edition, Addison Wesley, 2010

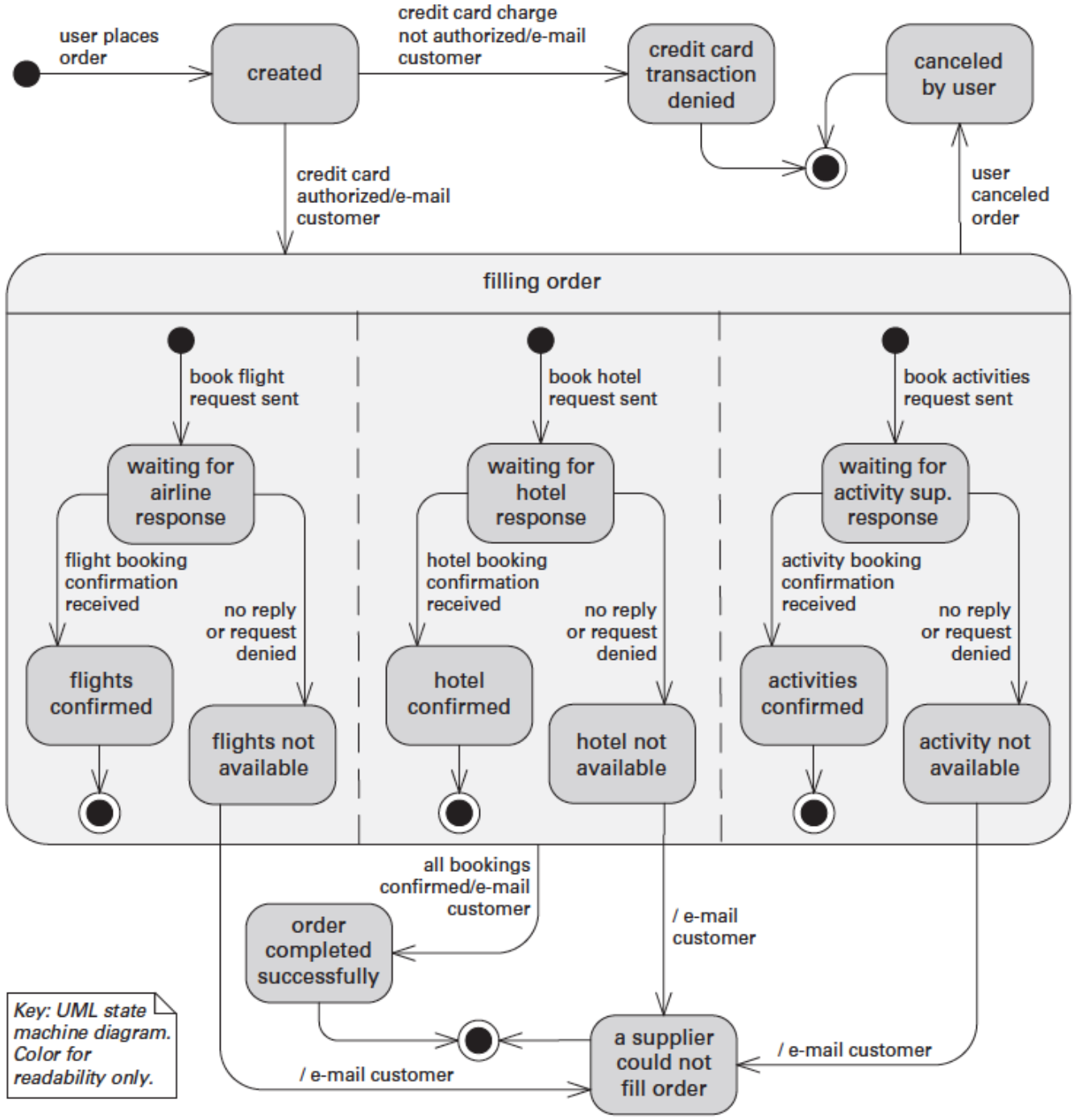
Notations for Documenting Behavior

- Comprehensive languages
  - *Comprehensive models* show the complete behavior of structural elements.
  - Given this type of documentation, it is possible to infer all possible paths from initial state to final state.
  - The state machine formalism represents the behavior of architecture elements because each state is an abstraction of all possible histories that could lead to that state.
  - State machine languages allow you to complement a structural description of the elements of the system with constraints on interactions and timed reactions to both internal and environmental stimuli.

State Machine



State Machine



From *Documenting Software Architectures: Views and Beyond*, 2nd edition, Addison Wesley, 2010

Documenting Quality Attributes

- Where do quality attributes show up in the documentation? There are five major ways:
  - Rationale that explains the choice of design approach should include a discussion about the quality attribute requirements and tradeoffs.
  - Architectural elements providing a service often have quality attribute bounds assigned to them, defined in the interface documentation for the elements, or recorded as *properties* that the elements exhibit.
  - Quality attributes often impart a "language" of things that you would look for. Someone fluent in the "language" of a quality attribute can search for the kinds of architectural elements) put in place to satisfy that quality attribute requirement.
  - Architecture documentation often contains a *mapping to requirements* that shows how requirements (including quality attribute requirements) are satisfied.
  - Every quality attribute requirement will have a constituency of stakeholders who want to know that it is going to be satisfied. For these stakeholders, the roadmap tells the stakeholder where in the document to find it.

Documenting Architectures That Change Faster Than You Can Document Them

- An architecture that changes at runtime, or as a result of a high-frequency release-and-deploy cycle, change much faster than the documentation cycle.
- Nobody will wait until a new architecture document is produced, reviewed, and released.
- In this case:
  - *Document what is true about all versions of your system*. Record those invariants as you would for any architecture. This may make your documented architecture more a description of constraints or guidelines that any compliant version of the system must follow.
  - *Document the ways the architecture is allowed to change*. This will usually mean adding new components and replacing components with new implementations. The place to do this is called the variability guide.

Documenting Architecture in an Agile Development Project

- Adopt a template or standard organization to capture your design decisions.
- Plan to document a view if (but only if) it has a strongly identified stakeholder constituency.

- Fill in the sections of the template for a view, and for information beyond views, when (and in whatever order) the information becomes available. But only do this if writing down this information will make it easier (or cheaper or make success more likely) for someone downstream doing their job.

- Don't worry about creating an architectural design document and then a finer-grained design document. Produce just enough design information to allow you to move on to code.

- Don't feel obliged to fill up all sections of the template, and certainly not all at once. Write "N/A" for the sections for which you don't need to record the information (perhaps because you will convey it orally).

- Agile teams sometimes make models in brief discussions by the whiteboard. Take a picture and use it as the primary presentation.

Summary

- You must understand the uses to which the writing is to be put and the audience for the writing.
-
- Architectural documentation serves as a means for communication among various stakeholders, not only up the management chain and down to the developers but also across to peers.
- An architecture is a complicated artifact, best expressed by focusing on views.
- You must choose the views to document, must choose the notation to document these views, and must choose a set of views that is both minimal and adequate.
- You must document not only the structure of the architecture but also the behavior.